Review

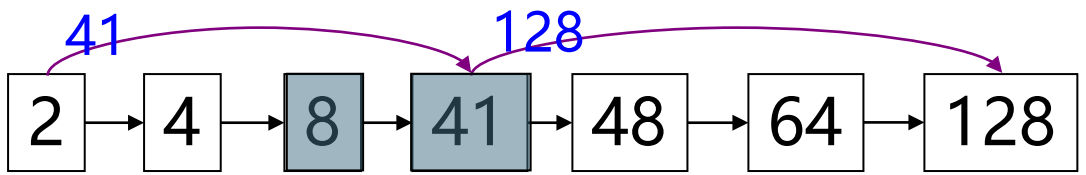
第二章 词项词典及倒排记录表

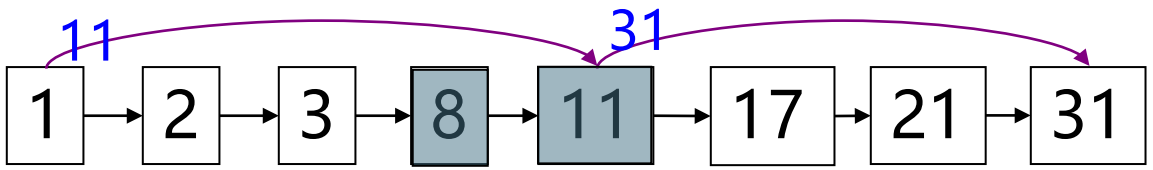
- ……
- 词干还原和词形归并
- 快速倒排表合并-跳表法
- 短语查询及位置索引
- 混合索引

其他词干还原工具(stemmer)

- **词干还原及其它归一化工作对检索的帮助**
  - 英语： 结果要一分为二，对某些查询来说提高了召回率，但是对另外一些查询来说降低了正确率
  - 比如, **operative**
  - **operate operating operates operation operative operatives operational**
  - **→ oper**
  - 对西班牙语、德语、芬兰语等语言非常有用
  - 其中对于芬兰语有30%的性能提高!

基于跳表指针的查询处理





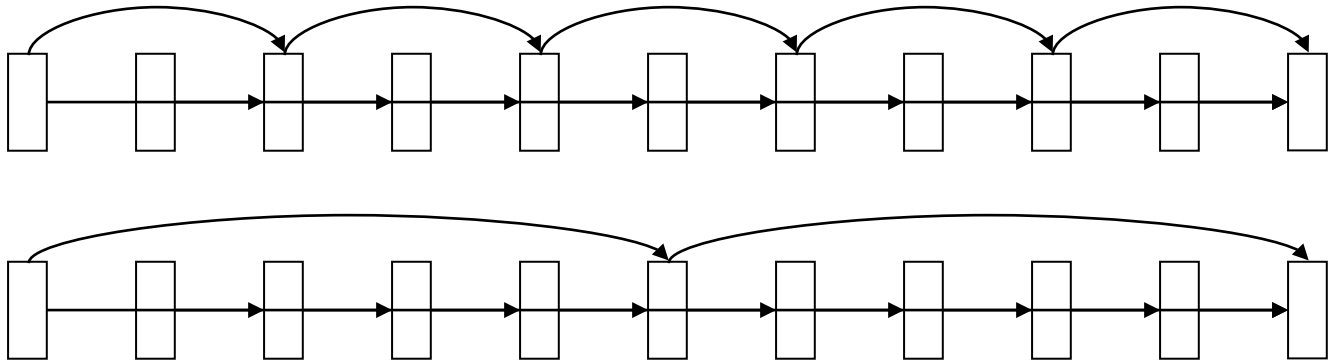
- 假定匹配到上下的指针都指向8，接下来两个指针都向下移动一位
- **比较41和11**，11小
- 此时看11上面的跳表指针，指向31，31仍然比41小，于是下指针可以直接跳过中间的11、17、21、31

基于跳表指针的倒排记录表合并算法

```
INTERSECTWITHSKIPS(p1, p2)
 1  answer ← ⟨ ⟩
 2  while p1 ≠ NIL and p2 ≠ NIL
 3  do if docID(p1) = docID(p2)
 4        then ADD(answer, docID(p1))
 5             p1 ← next(p1)
 6             p2 ← next(p2)
 7        else if docID(p1) < docID(p2)
 8                then if hasSkip(p1) and (docID(skip(p1)) ⩽ docID(p2))
 9                     then while hasSkip(p1) and (docID(skip(p1)) ⩽ docID(p2))
10                          do p1 ← skip(p1)
11                     else p1 ← next(p1)
12                else if hasSkip(p2) and (docID(skip(p2)) ⩽ docID(p1))
13                     then while hasSkip(p2) and (docID(skip(p2)) ⩽ docID(p1))
14                          do p2 ← skip(p2)
15                     else p2 ← next(p2)
16  return answer
```

跳表指针的位置

- **指针数目过多过少都不合适**，要有一个均衡性
  - 指针越多→ 跳步越短 ⇒ 更容易跳转，但是需要更多的与跳表指针指向记录的比较
  - 指针越少→ 比较次数越少，但是跳步越长 ⇒ 成功跳转的次数少



双词(Biword)索引--更长的短语查询处理

- 例子： stanford university palo alto
- 处理方法：将其拆分成**基于双词的布尔查询式**
  - stanford university AND university palo AND palo alto
- **2-gram(K-gram)**
  - 一个K-gram代表由K个字符组成的序列
- 如果不检查文档，无法确认满足上述表达式的文档是否真正满足上述短语查询。也就是说满足上述布尔表达式只是满足短语查询的**充分条件**

扩展的双词（Extended Biword）

- 对待索引文档进行**词性标注**
- 将词项进行组块，每个组块包含**名词 (N) 和冠词/介词 (X)**
- **称具有NX*N形式的词项序列为扩展双词**(extended biword)
  - 将这样扩展词对作为词项放入词典中
- 例子：catcher in the rye (书名： 麦田守望者)
  - N X X N
- **查询处理：将查询也分析成N和X序列**
  - 将查询切分成扩展双词
  - 在索引中查找: catcher rye

位置索引的例子

- 对于输入的短语查询，需要在文档的层次上进行**迭代(不同位置上)合并**
- 不仅仅简单合并，还要考虑位置匹配

<*be*: 993427;
1: 7, 18, 33, 72, 86, 231;
2: 3, 149;
4: 17, 191, 291, 430, 434;
5: 363, 367, ...>



短语查询的处理

- 短语查询： “to be or not to be”
- **对每个词项，抽出其对应的倒排记录表**: to, be, or, not.
- 合并<docID:位置 >表，考虑 “to be or not to be”

- to:
  - 2:1,17,74,222,551;
  - 4:8,16,190,429,433;
  - 7:13,23,191; ...
- be:
  - 1:17,19;
  - 4:17,191,291,430,434;
  - 5:14,19,101; ...

混合索引

- **上述两种索引方式(双词索引、位置索引)可以混合使用**
  - 处理开销最大的短语查询往往是这样一些短语：**每个词都非常常见，但是组合起来却相对很少见**
  - 某些特定的短语采用位置索引的方式那么效率不高
  - “Michael Jackson”，“Britney Spears”
  - “Air Jordan”，“Macbook Air”
  - “The Who”（英国一著名摇滚乐队），采用位置索引，效率更低

章小结

- 索引构建过程(特别是预处理)
- 如何对索引文档进行处理来得到词典
  - **理解文档(document)的概念**
  - **词条化(Tokenization)，理解词条(token)的概念**
  - **词项生成，理解词项(term)的概念**
- 倒排记录表
  - **更快的合并算法: 跳表法(skip list)**
  - **短语查询的处理及带位置信息的倒排索引**

第3讲 索引构建

Index construction

提纲

1. 上一讲回顾
2. 简介
3. BSBI算法
4. SPIMI算法
5. 分布式索引构建
6. 动态索引构建

章节概述-In general

- **计算机硬件基础知识**
- **两种索引构建算法**
  - **BSBI(简单)**
  - **SPIMI(更符合实际情况)**
- **分布式索引构建: MapReduce**
- **动态索引构建: 如何随着文档集变化更新索引**
- **其他索引的讨论：排序、安全性**

章节概述-In detail

- **与索引构建相关的计算机硬件的基本知识**
- **一种面向静态文档集的高效单机索引算法——基于块的排序索引构建算法BSBI**
  - 基于排序的基本索引算法
  - 一个更具扩展性的版本
- **内存式单遍扫描索引构建算法SPIMI**
  - 并不将词汇表都加载到内存中
  - 更具扩展性

章节概述-In detail

- **分布式索引构建**
  - 像Web一样的大规模的文档集合
  - 考虑如何在成百上千台计算机构成的计算机集群上进行分布式索引构建
- **动态索引构建**
  - 很多文档集会动态变化
  - 以便将文档集的变化即时反映到索引中
- **索引构建中可能会遇到的一些复杂情况**
  - 排序式检索中的索引问题
  - 安全性

Preface

索引构建

- 如何建立倒排索引，这个过程称为**索引构建**
- **index construction或indexing**

索引器

- 构建索引的程序或计算机称为**索引器**
- **indexer**

Preface

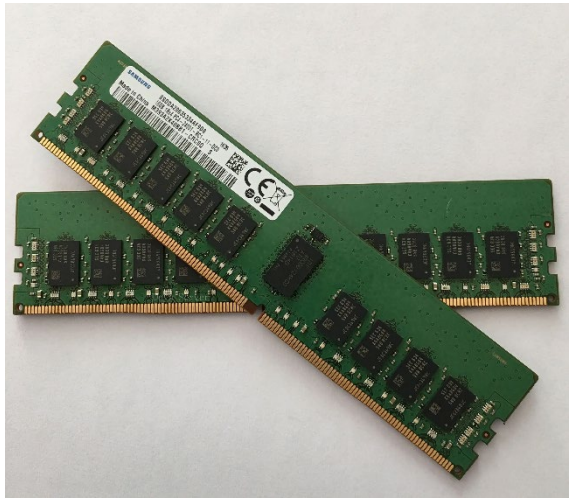
- **索引器**需要原始文本，但是文档可能会采用各种**编码格式**
- 索引器对中间文件和最后的索引文件进行**压缩或者解压缩**
- 在Web搜索中，文档往往并不来自于本地，而必须要通过**网络采集**才能得到
- 在企业搜索中，很多文档内嵌在各种**内容管理系统、邮件系统或者数据库**中

硬件基础知识

- 信息检索系统中的很多设计上的决策取决于**硬件限制**
- 本课程中需要用到的**硬件知识**
  - **内存 (Memory)**
    - **结构：单通道、双通道**
  - **硬盘 (Hard Disk)**
    - **类型：机械、SSD、NVMe接口等**
- **扩展：手机的“内存”**
  - **运行内存(RAM)**
  - **可用内存(ROM)**

硬件基础知识

- **在内存中访问数据会比从硬盘访问数据快很多(大概10倍左右的差距)**
  - **只需要几个时钟周期（大概5×10^{-9} s）便可以访问内存中的一个字节**
  - **从磁盘传输一个字节所需要的时间则长得多（大概2×10^{-8} s）**
- 尽可能地把数据放在内存中，特别是那些访问频繁的数据
  - 频繁访问的磁盘数据放入内存的技术称为**缓存技术（caching）**





硬件基础知识

- **寻道时间**
  - 定义：进行磁盘读写时，磁头移到数据所在的磁道需要的一段时间
  - 硬盘寻道时间是闲置时间：**磁头在定位时不发生数据传输**
  - **对典型的磁盘来说平均在5ms左右**
  - 为使数据传输率最大，连续读取的数据块也应该在磁盘上连续存放





硬件基础知识

- **寻道时间**
  - **为优化从磁盘到内存的传送时间，一个大(连续)块的传输会比多个小块(非连续)的传输速度快**
  - 例如
    - **大概只需要0.2秒钟就可以将一个连续存放的10MB数据块从磁盘传输到内存**
    - **如果上述数据存放在100个非连续的块中，需要移动100次磁头**

硬件基础知识

- **硬盘I/O基于块**
  - 读写时是整块进行的
  - 从磁盘读取一个字节和读取一个数据块所耗费的时间可能一样多
  - **块大小：8KB到256KB不等**
  - 内存中保存读写块的区域称为**缓冲区（buffer）**

硬件基础知识

- **数据从磁盘传输到内存是由系统总线而不是处理器来实现的**
  - 磁盘I/O时**处理器**仍然可以处理数据
  - 可以利用该特点加速数据的传输过程
  - **比如将数据进行压缩然后再存储在磁盘上**
  - 假定采用一种**高效的解压缩算法**，那么读磁盘压缩数据再解压所花的**时间**往往会比直接读取未压缩数据的**时间**要少

硬件基础知识

服务器

- IR系统的**服务器的典型配置**是几个GB的内存，有时内存可能达到几十GB，数百G或者上T的硬盘（**2007年**）

容错处理

- **容错处理**的代价非常昂贵：采用**多台普通机器**会比一台提供容错的机器的价格更便宜

一些统计数据(ca. 2008)

一个**2007年度**的典型计算机系统的参数

符号	含义	值
s	平均寻道时间	5 ms = 5×10^{-3} s
b	每个字节的传输时间	0.02 μs = 2×10^{-8} s
	处理器时钟频率	10^{9} s^{-1}
P	底层操作时间 (e.g., 如word的比较和交换)	0.01 μs = 10^{-8} s
	内存大小	几GB
	磁盘大小	1 TB或更多

提纲

1. 上一讲回顾
2. 简介
3. BSBI算法
4. SPIMI算法
5. 分布式索引构建
6. 动态索引构建

索引构建过程: 词条序列

- <词条，docID>二元组

Doc 1

I did enact Julius Caesar I was killed i' the Capitol; Brutus killed me.

Doc 2

So let it be with Caesar. The noble Brutus hath told you Caesar was ambitious



Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

建立不包含位置信息的索引

索引构建过程: 排序

- **排序**
  1. **按词项**
  2. **每个词项按docID**

索引构建的核心步骤

Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2



Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2

索引构建过程: 词典 & 倒排记录表

- **某个词项在单篇文档中的多次出现会被合并**
- 拆分成词典和倒排记录表两部分
- 每个词项出现的**文档数目**(doc. frequency, DF)会被加入

Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2



term	doc. freq.	→	postings lists
ambitious	1	→	2
be	1	→	2
brutus	2	→	1 → 2
capitol	1	→	1
caesar	2	→	1 → 2
did	1	→	1
enact	1	→	1
hath	1	→	2
i	1	→	1
i'	1	→	1
it	1	→	2
julius	1	→	1
killed	1	→	1
let	1	→	2
me	1	→	1
noble	1	→	2
so	1	→	2
the	2	→	1 → 2
told	1	→	2
you	1	→	2
was	2	→	1 → 2
with	1	→	2

构建准备

- **为使索引构建过程效率更高，将词项用其ID来代替**
- **每个词项的ID是唯一的序列编号，将词项映射成其ID**
- **本章介绍的索引构建方法均采用单遍扫描方式**
- 一种**两遍扫描**的方法
  - 第一遍扫描得到词汇表，第二遍扫描才构建倒排索引

一篇Reuters RCV1文档的样例

- **《莎士比亚全集》规模较小，用来构建索引不能说明问题**
- **本讲使用Reuters RCV1文档集来介绍可扩展的索引构建技术**
- **路透社1995到1996年一年的英语新闻报道**



一篇Reuters RCV1文档的样例

- **大概有800 000篇文档**
- **忽略其中的图片等多媒体信息**
- **而只关注其中的文本信息**



REUTERS

You are here: Home > News > Science > Article

Go to a Section: U.S. International Business Markets Politics Entertainment Technology Sports Oddly En

Extreme conditions create rare Antarctic clouds

Tue Aug 1, 2006 3:20am ET

Email This Article | Print This Article | Reprin

[-] Text [-

SYDNEY (Reuters) - Rare, mother-of-pearl colored clouds caused by extreme weather conditions above Antarctica are a possible indication of global warming, Australian scientists said on Tuesday.

Known as nacreous clouds, the spectacular formations showing delicate wisps of colors were photographed in the sky over an Australian

Reuters RCV1语料库的统计信息

N	文档数目	800,000
L	每篇文档的词条数目	200
M	词项数目(= 词类数目)	400,000
	每个词条的字节数 (含空格和标点)	6
	每个词条的字节数 (不含空格和标点)	4.5
	每个词项的字节数	7.5
T	无位置信息索引中的倒排记录数目	100,000,000

Reuters RCV1语料库的统计信息

N	**文档数目**	**800,000**
L	**每篇文档的词条数目**	**200**
M	**词项数目(= 词类数目)**	**400,000**
	每个词条的字节数 (含空格和标点)	**6**
	每个词条的字节数 (不含空格和标点)	**4.5**
	每个词项的字节数	**7.5**
T	**无位置信息索引中的倒排记录数目**	**100,000,000**

练习:

(1)一个词项的平均出现次数是多少？即一个词项平均对应几个词条?

(2) 词条总数是多少？所有词条占的空间大小?

Reuters RCV1语料库的统计信息

N	**文档数目**	**800,000**
L	**每篇文档的词条数目**	**200**
M	**词项数目(= 词类数目)**	**400,000**
	每个词条的字节数 (含空格和标点)	**6**
	每个词条的字节数 (不含空格和标点)	**4.5**
	每个词项的字节数	**7.5**
T	**无位置信息索引中的倒排记录数目**	**100,000,000**

(1) 一个词项的平均出现次数是多少？即一个词项平均对应几个词条?

每篇文档的词项数目：

每篇文档的词条数目：

词项与词条的比例：

Reuters RCV1语料库的统计信息

N	**文档数目**	**800,000**
L	**每篇文档的词条数目**	**200**
M	**词项数目(= 词类数目)**	**400,000**
	每个词条的字节数 (含空格和标点)	**6**
	每个词条的字节数 (不含空格和标点)	**4.5**
	每个词项的字节数	**7.5**
T	**无位置信息索引中的倒排记录数目**	**100,000,000**

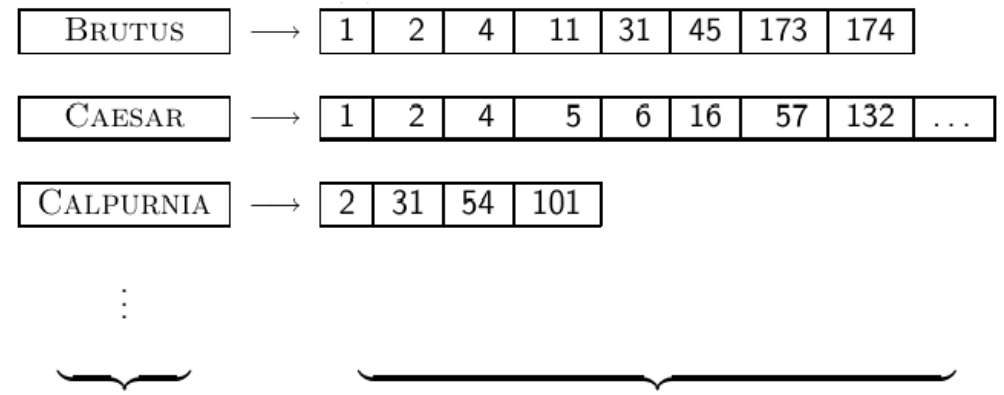
(2) 词条总数是多少？所有词条占的空间大小?

词项ID-文档ID对的数目：每篇文档的词条数目*文档数目

基于排序的索引构建方法

- **如果每条倒排记录占10–12个字节，那么对于大规模语料，需要更大的存储空间**
- 以Reuters-RCV1语料为例
  - **倒排记录1亿个词条 (T = 100,000,000)**
  - **词项ID-文档ID对的数目约为1.6亿条**
  - **假设每个词项ID和文档ID各占4B**
  - **词项ID-文档ID对需要1.28GB存储空间**

基于排序的索引构建方法



词典　　倒排记录表

- **在构建索引时，每次分析一篇文档**
- **对于每个词项而言，其倒排记录表不到最后一篇文档都是不完整的**
- 那么能否在最后排序之前将前面产生的倒排记录表全部放在内存中？
- 答案显然是否定的，特别是对大规模的文档集来说

基于排序的索引构建方法

- 以Reuters-RCV1语料为例
  - **词项ID-文档ID对需要1.28GB存储空间**
  - **倒排记录表**
  - 可以放在2010年的一台典型配置的计算机的内存中
- 但是这种**基于内存的索引构建方法**显然无法扩展到**大规模文档集**上
- 因此，需要在**磁盘上存储中间结果**

是否在磁盘上采用同样的算法？

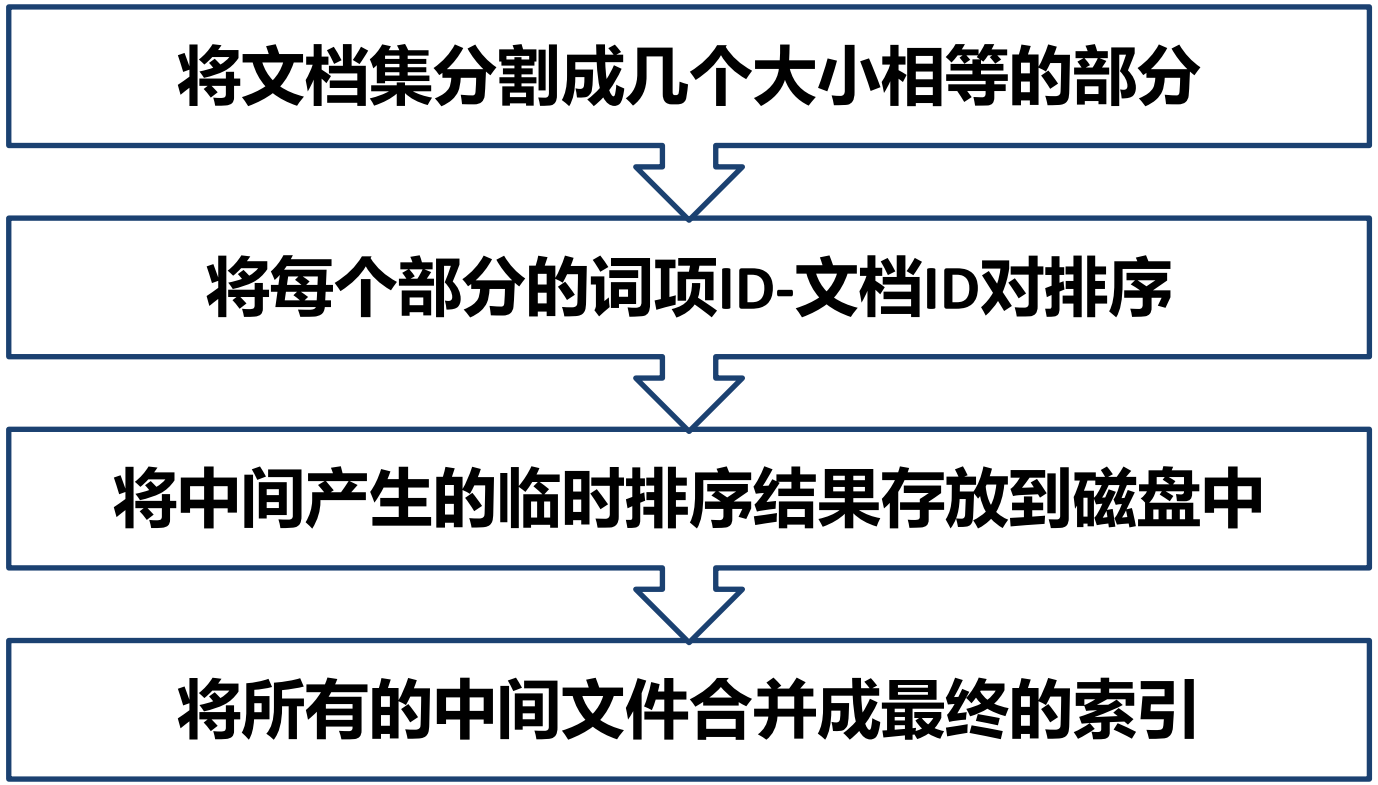
- 能否使用前面同样的算法，但是在磁盘而不是内存中完成排序?
  - 不可能，这是因为对 $T = 100,000,000$条记录**在磁盘上进行排序需要太多的磁盘寻道过程**
- 由于内存不足，我们必须使用**基于磁盘的外部排序算法（external sorting algorithm)**
- 为了达到可以接受的速度，对该算法的核心要求
  - **在排序时尽量减少磁盘随机寻道的次数**
- 磁盘**顺序读取**速度会比**随机寻道**速度快得多

外部排序算法中磁盘寻道次数很少

- 需要对T = 100,000,000条无位置信息的**倒排记录进行排序**
  - 每条倒排记录需要**12字节** (4+4+4: termID, docID, df)
- 定义一个能够包含10,000,000条上述倒排记录的**数据块**
  - 这个数据块很容易放入内存中(**12*10M=120M**)
  - 对于RCV1有10个数据块

基于块的排序索引构建算法BSBI
（Blocked Sort-Based Indexing）

算法的基本思路



基于块的排序索引构建算法BSBI（Blocked Sort-Based Indexing）

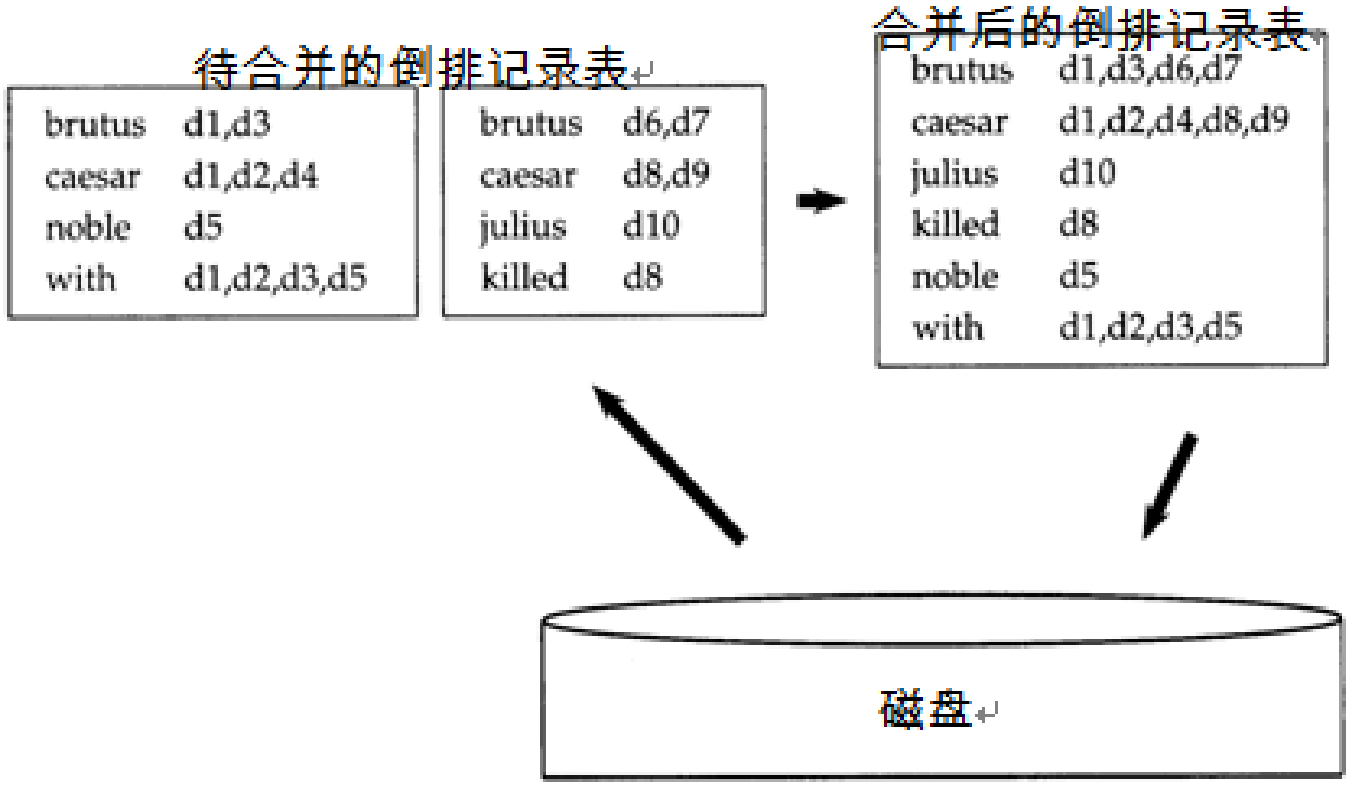
```
BSBIndexConstruction()
1  n ← 0
2  while (all documents have not been processed)
3  do n ← n + 1
4     block ← ParseNextBlock()
5     BSBI-Invert(block)
6     WriteBlockToDisk(block, f_n)
7  MergeBlocks(f_1, ..., f_n; f_merged)
```

- 将10个块索引同时合并成一个索引文件
- 该算法中有一个关键决策就是确定块的大小

算法的基本思路

- 对每个块
  - (i) 倒排记录累积到10,000,000条
  - (ii) 在内存中排序
  - (iii) 写回磁盘
- 最后将所有的块合并成一个大的有序的倒排索引

两个块的合并过程



- **合并时，同时打开所有块对应的文件**
- 内存中维护了
  - 为10个块准备的**读缓冲区**
  - 一个为最终合并索引准备的**写缓冲区**
- **每次迭代中，利用优先级队列（即堆结构）或者类似的数据结构选择最小的未处理词项ID进行处理**
- 读入该词项的倒排记录表并**合并**，合并结果写回磁盘中
- 需要时，再次从文件中读入数据到每个**读缓冲区**

基于块的排序索引构建算法BSBI：时间复杂度

- 由于该算法最主要的时间消耗在**排序**上
  - 因此其**时间复杂度为Θ（TlogT）**
- 其中T是**所需要排序的项数目的上界**（即**词项ID-文档ID对的个数**）
- **实际的索引构建时间**往往取决于以下两者时间
  - **文档分析（PARSENEXTBLOCK）**
  - **最后合并（MERGEBLOCKS）**

1台机器下采用**基于块的排序索引构建算法**BSBI

对**Google级规模数据**构建索引的时间

symbol	statistic	value
s	average seek time	5 ms $= 5 \times 10^{-3}$ s
b	transfer time per byte	0.02 μs $= 2 \times 10^{-8}$ s
	processor's clock rate	10^9 s^{-1}
p	lowlevel operation	0.01 μs $= 10^{-8}$ s
	number of machines	1
	size of main memory	8 GB
	size of disk space	unlimited
N	documents	10^{11} (on disk)
L	avg. # word tokens per document	10^3
M	terms (= word types)	10^8
	avg. # bytes per word token (incl. spaces/punct.)	6
	avg. # bytes per word token (without spaces/punct.)	4.5
	avg. # bytes per term (= word type)	7.5

符号	含义
s	平均寻道时间
b	每个字节的传输时间
	处理器时钟频率
P	底层操作时间 (e.g., 如word的比较和交换)
	主机数量
	内存大小
	磁盘大小
N	文档总数
L	每篇文档的平均词条数目
M	词项数目
	每个词条的平均字节数（含空格和标点符号）
	每个词条的平均字节数（不含空格和标点符号）
	每个词项的平均字节数

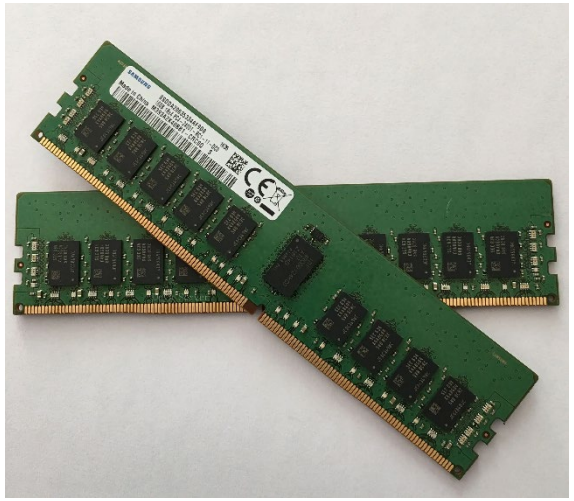
Review

第三章 索引构建

- **与索引构建相关的计算机硬件的基本知识**
- **一种面向静态文档集的高效单机索引算法——基于块的排序索引构建算法BSBI**
  - 基于排序的基本索引算法
  - 一个更具扩展性的版本
- **内存式单遍扫描索引构建算法SPIMI**
  - ……

硬件基础知识

- **在内存中访问数据会比从硬盘访问数据快很多(大概10倍左右的差距)**
  - **只需要几个时钟周期（大概5 × 10^{-9} s）便可以访问内存中的一个字节**
  - **从磁盘传输一个字节所需要的时间则长得多（大概2 × 10^{-8} s）**
- 尽可能地把数据放在内存中，特别是那些访问频繁的数据
  - 频繁访问的磁盘数据放入内存的技术称为**缓存技术（caching）**





硬件基础知识

- **硬盘I/O基于块**
  - 读写时是整块进行的
  - 从磁盘读取一个字节和读取一个数据块所耗费的时间可能一样多
  - **块大小：8KB到256KB不等**
  - 内存中保存读写块的区域称为**缓冲区（buffer）**

Reuters RCV1语料库的统计信息

N	**文档数目**	**800,000**
L	**每篇文档的词条数目**	**200**
M	**词项数目(= 词类数目)**	**400,000**
	每个词条的字节数 (含空格和标点)	**6**
	每个词条的字节数 (不含空格和标点)	**4.5**
	每个词项的字节数	**7.5**
T	**无位置信息索引中的倒排记录数目**	**100,000,000**

练习:

(1)一个词项的平均出现次数是多少？即一个词项平均对应几个词条?

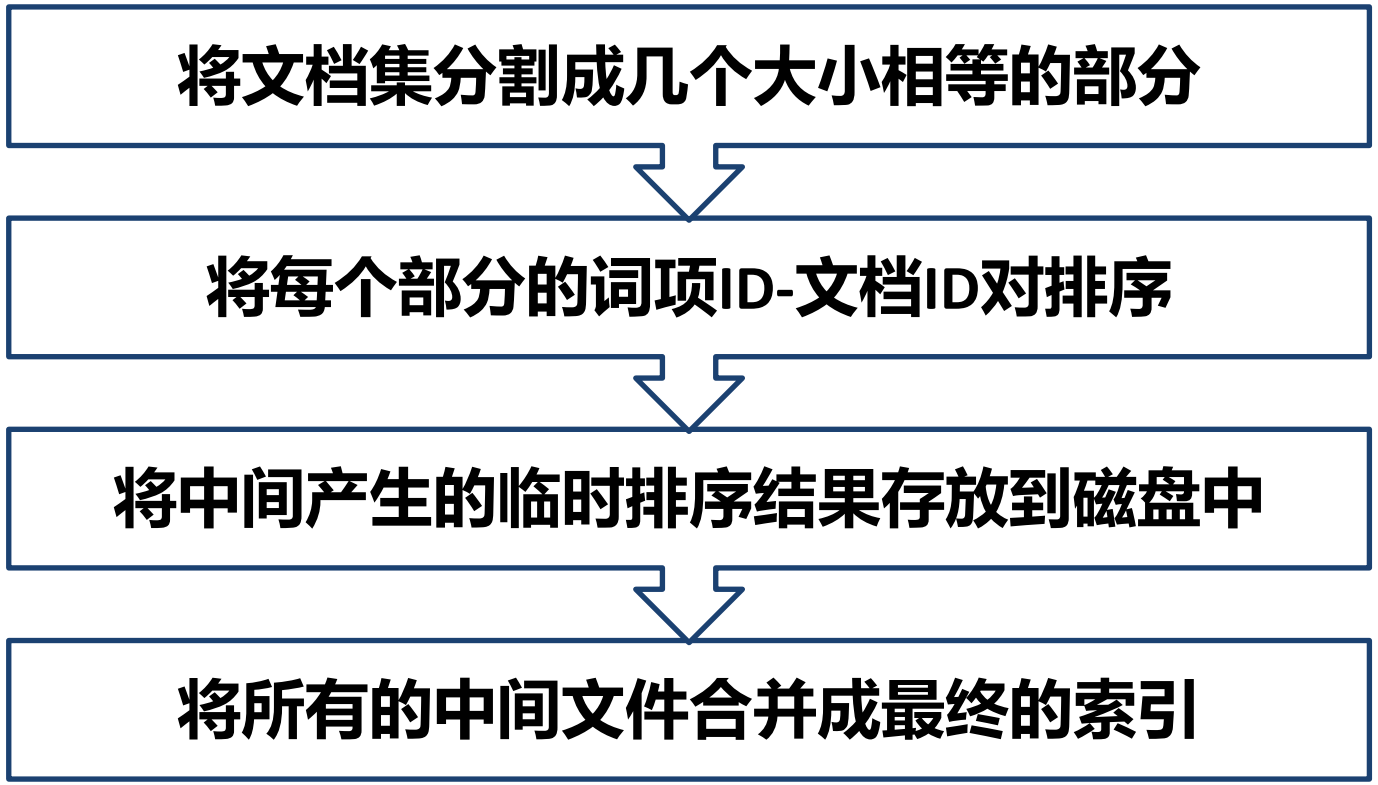
(2) 词条总数是多少？所有词条占的空间大小?

外部排序算法中磁盘寻道次数很少

- 需要对T = 100,000,000条无位置信息的**倒排记录进行排序**
  - 每条倒排记录需要**12字节** (4+4+4: termID, docID, df)
- 定义一个能够包含10,000,000条上述倒排记录的**数据块**
  - 这个数据块很容易放入内存中(**12*10M=120M**)
  - 对于RCV1有10个数据块

基于块的排序索引构建算法BSBI
（Blocked Sort-Based Indexing）

算法的基本思路



基于块的排序索引构建算法BSBI（Blocked Sort-Based Indexing）

```
BSBIndexConstruction()
1  n ← 0
2  while (all documents have not been processed)
3  do n ← n + 1
4     block ← ParseNextBlock()
5     BSBI-Invert(block)
6     WriteBlockToDisk(block, f_n)
7  MergeBlocks(f_1, ..., f_n; f_merged)
```

- 将10个块索引同时合并成一个索引文件
- 该算法中有一个关键决策就是确定块的大小

算法的基本思路

- 对每个块
  - (i) 倒排记录累积到10,000,000条
  - (ii) 在内存中排序
  - (iii) 写回磁盘
- 最后将所有的块合并成一个大的有序的倒排索引

提纲

1. 上一讲回顾
2. 简介
3. BSBI算法
4. SPIMI算法
5. 分布式索引构建
6. 动态索引构建

基于排序的索引构建算法的问题

- 假定词典可以在内存中放下
  - 通常需要一部词典(动态增长)来将term映射成termID
- 实际上，倒排记录表可以直接采用term, docID方式而不是termID, docID方式
- BUT, 此时中间文件将会变得很大

内存式单遍扫描索引构建算法SPIMI

(Single-pass in-memory indexing)

- **关键思想**
  - **对每个块都产生一个独立的词典**
    - 不需要在块之间进行term-termID的映射
  - **对倒排记录表不排序，按照出现的先后顺序排列**
- 基于上述思想可以对**每个块**生成一个完整的倒排索引
- 这些独立的索引最后合并一个**大索引**

SPIMI算法中块倒排索引的生成

- 算法逐一处理（Line 4）每个词项-文档ID对
- 如果词项是出现则直接返回其倒排记录表（Line 7）第一次出现，那么将之加入词典（最好通过哈希表来实现），同时建立一个新的倒排记录表（Line 6）
- 如果该词项不是第一次出现，则直接返回其倒排记录表（Line 7）

```
SPIMI-INVERT(token_stream)
 1  output_file ← NEWFILE()
 2  dictionary ← NEWHASH()
 3  while (free memory available)
 4  do token ← next(token_stream)
 5     if term(token) ∉ dictionary
 6        then postings_list ← ADDTODICTIONARY(dictionary, term(token))
 7        else postings_list ← GETPOSTINGSLIST(dictionary, term(token))
 8     if full(postings_list)
 9        then postings_list ← DOUBLEPOSTINGSLIST(dictionary, term(token))
10     ADDTOPOSTINGSLIST(postings_list, docID(token))
11  sorted_terms ← SORTTERMS(dictionary)
12  WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13  return output_file
```

Merging of blocks is analogous to BSBI.

SPIMI算法中块倒排索引的生成

```
SPIMI-INVERT(token_stream)
 1  output_file ← NEWFILE()
 2  dictionary ← NEWHASH()
 3  while (free memory available)
 4  do token ← next(token_stream)
 5     if term(token) ∉ dictionary
 6        then postings_list ← ADDTODICTIONARY(dictionary, term(token))
 7        else postings_list ← GETPOSTINGSLIST(dictionary, term(token))
 8     if full(postings_list)
 9        then postings_list ← DOUBLEPOSTINGSLIST(dictionary, term(token))
10     ADDTOPOSTINGSLIST(postings_list, docID(token))
11  sorted_terms ← SORTTERMS(dictionary)
12  WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13  return output_file
Merging of blocks is analogous to BSBI.
```

- 省略：
  - 文档分析
  - 将文档转换成词项-文档ID流（token_stream）的过程
- 反复调用SPIMI-INVERT函数
  - 直到将全部的文档集处理完为止

SPIMI算法的优点

- 由于不需要排序操作，因此处理的速度更快
- 保留了倒排记录表对词项的归属关系
  - 能够节省内存
  - 词项的ID也不需要保存
- 每次单独的SPIMI-INVERT调用能够处理的**块大小**可以非常大
  - 整个倒排索引的构建过程也因此会非常高效

SPIMI算法的优点

- 由于事先并不知道每个词项的倒排记录表大小，算法一开始会分配一个较小的倒排记录表空间
- 每次**当该空间放满的时候，就会申请加倍的空间**（L8 ~ L9）

```
8      if full(postings_list)
9        then postings_list ← DoublePostingsList(dictionary, term(token))
```

- 一些空间被浪费
  - 正好抵消了不用保存词项ID所省下的空间

SPIMI算法中块倒排索引的生成

- 总体来说，**在SPIMI中对块建立索引所需的内存空间比BSBI少**
- 当内存耗尽时，包括词典和倒排记录表的块索引将被写到磁盘上（L12）

```
11  sorted_terms ← SortTerms(dictionary)
12  WriteBlockToDisk(sorted_terms, dictionary, output_file)
```

- 在这之前，为使倒排记录表按照词典顺序排序来加快最后的合并过程，要对词项进行**排序操作**（L11）
  - 如果每个块的倒排记录表没有事先排好序，那么合并过程很难通过一个简单的逐块扫描算法来实现
- 每次对SPIMI-Inverter的调用都会**写一个块到磁盘**，和BSBI一样
  - SPIMI 最后一步就是将**多个块合并成最后的倒排索引**

BSBI与SPIMI算法比较

- **基于块的排序索引构建算法**BSBI
  - 一开始就整理出所有的**词项ID-文档ID**并对它们进行排序
- **内存式单遍扫描索引构建算法**SPIMI
  - 直接在倒排记录表中增加一项（L10）
  - 倒排记录表**动态增长**
  - 立即就可以实现全体倒排记录表的收集

```
10    AddToPostingsList(postings_list, docID(token))
```

内存式单遍扫描索引构建算法SPIMI:

压缩

- 如果使用压缩，SPIMI将更加高效
  - **词项的压缩**
  - **倒排记录表的压缩**
- 参见下一章节

内存式单遍扫描索引构建算法SPIMI:

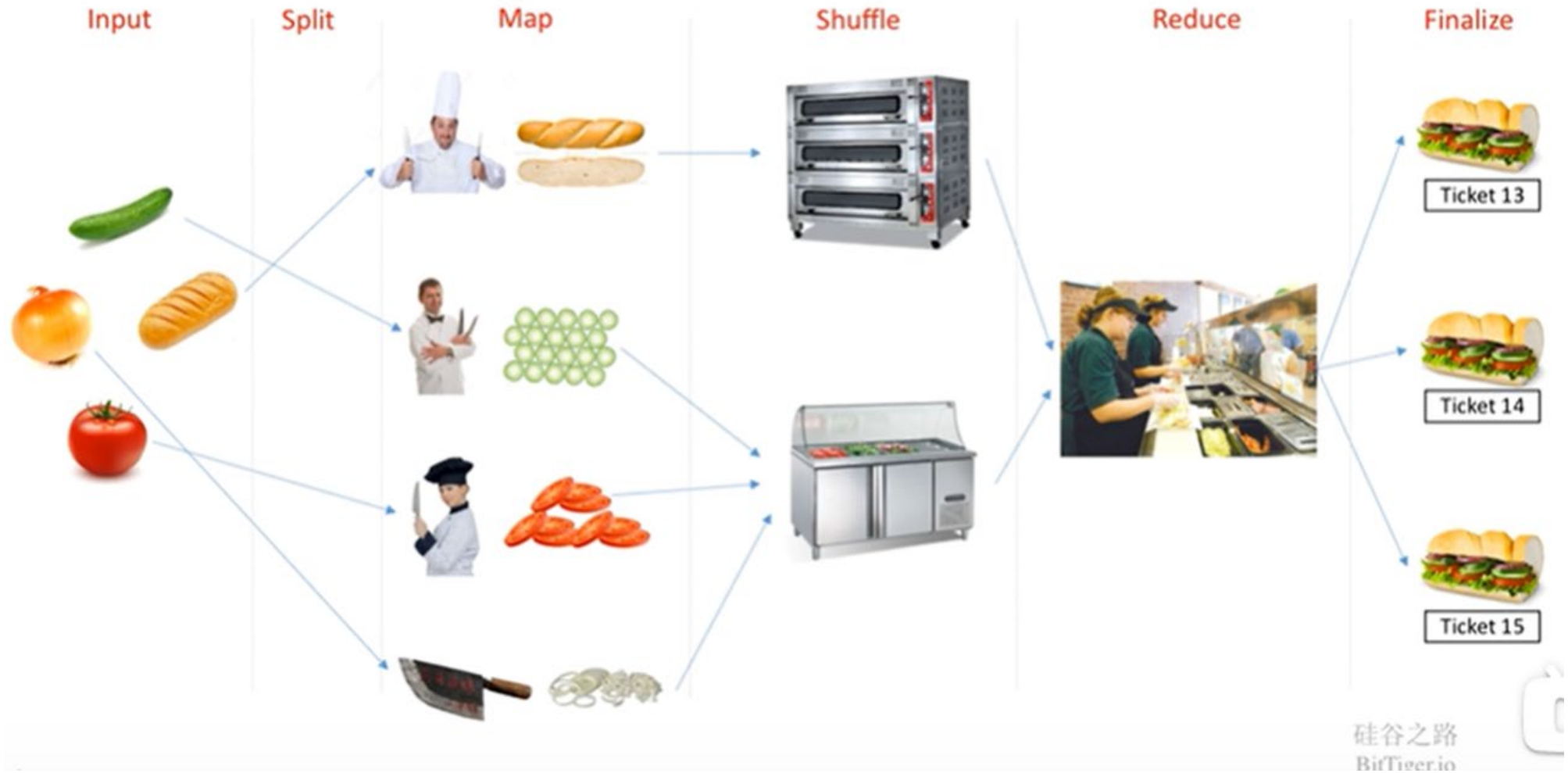
时间复杂度

- **SPIMI算法的时间复杂度是Θ(T)**
  - **T是所需要排序的项数目的上界（词项ID-文档ID对的个数）**
- **不需要对词项-文档ID对进行排序操作**
- 所有操作最多和**文档集大小**成线性关系

提纲

1. 上一讲回顾
2. 简介
3. BSBI算法
4. SPIMI算法
5. 分布式索引构建
6. 动态索引构建

A case


Input
Split
Map
Shuffle
Reduce
Finalize
Ticket 13
Ticket 14
Ticket 15
硅谷之路
BitTiger.io

分布式索引构建

- 对于Web数据级别的数据建立
  - 必须使用**分布式计算机集群**
- **单台**机器都是有可能出现故障的
  - 可能突然慢下来或者失效，不可事先预知
- 如何使用**一批**机器?

分布式索引构建

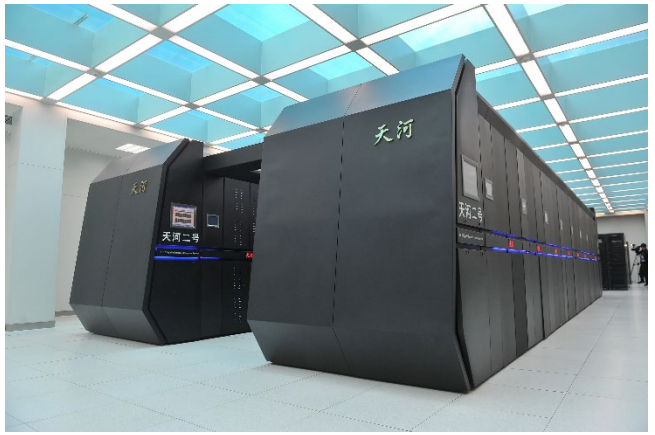
- 对Web构建规模合理的索引常常需要大规模的**计算机集群**
- cluster指的是一组**紧耦合**共同工作的计算机
- 因此，Web搜索引擎通常使用**分布式索引构建**（distributed indexing）算法构建索引
- 其索引结果也是分布式的，它往往按照**词项或文档进行分割**后分布在多台计算机上

分布式索引构建

- 本节介绍的分布式索引构建方法是**MapReduce**的一个应用
- MapReduce是一个**通用的分布式计算架构，面向大规模计算机集群**
- 集群的关键是，利用价格低廉的日常用的计算机（称为**节点，node**）来解决大型的计算问题
- 这些计算机都采用**通用的标准部件**（处理器、内存和磁盘），而不是象**超级计算机**那样采用专用硬件

超级计算机





- 极大的数据存储容量
- 极快速的数据处理速度
- 专用处理器
- 特定用途(气候预测、天体演变模拟、地震模拟等)





分布式索引构建

- 尽管在这样的一个**计算机集群**当中包含成百上千台计算机，但**每台计算机都有可能在任意时刻失效**
- 因此，要保障分布式索引构建过程的**健壮性**，就必须把整个任务分成易分配的子任务块，并在节点失效时能够重新分配
- 集群中的**主控节点（master node）**负责处理任务在工作节点上的分配和重分配

分布式索引构建

- **健壮性、鲁棒性**
  - **Robust的音译，健壮和强壮**
  - **是在异常和危险情况下系统生存的能力**
- 比如说，计算机软件在输入错误、磁盘故障、网络过载或有意攻击情况下，能否不死机、不崩溃，就是该软件的鲁棒性

Google 数据中心(2007 estimates; Gartner)

- Google数据中心主要都是**普通机器**
- **数据中心均采用分布式架构，在世界各地分布**
  - 100万台服务器，300个处理器/核
  - Google每15分钟装入100,000个服务器
- **每年的支出大概是每年2-2.5亿美元**
- **这可能是世界上计算能力的10%!**

Google 数据中心(2007 estimates; Gartner)

- 在一个1000个节点组成的无容错系统中，每个节点的正常运行概率为99.9%，那么整个系统的正常运行概率是多少?
- **答案: 63%**
- 假定一台服务器3年后会失效，那么对于100万台服务器，机器失效的平均间隔大概是多少?
- **答案: 不到2分钟**

分布式索引构建

- **维持一台**被认为是安全的**主机(Master)指挥索引构建任务**
- **将索引划分成多组并行任务**
- 主机将把每个任务分配给某个缓冲池中的**空闲机器**来执行

分布式索引构建

- MapReduce中的Map、Reduce过程将计算任务划分成子任务块，以便每个工作节点在短时间内快速处理
- Map：映射
- 归纳与演绎
  - Reduce
  - Deduce

分布式索引构建

- 首先，输入数据（网页集合）被分割成**n个数据片（split）**，数据片大小的选择一定要保证任务的均匀、高效分布，同时每个数据片不能太大，数据片的数目也不能太多
- 在分布式索引中，数据片的大小采用16MB或者64MB
  - 效果通常不错
- **各个数据片并不预先分配给各台计算机，而是在运行过程中由主控节点分配**
- 一旦一台计算机**完成**了某个数据片的处理任务，主控节点就会分配下一个数据片给它进行处理
  - 如果某台机器**宕机或者由于硬件问题**导致机器变慢，它上面运行的任务数据片就可以重新分配给其他计算机

并行任务

- 一般来说，MapReduce会通过**键-值对（key-value pair）**的转换处理，将一个大型的计算问题转化成较小的子问题
- 在**索引构建**中，**键-值对的形式就是（词项ID, 文档ID）**
- 在分布式索引构建过程中，从**词项到其ID**的映射同样要分布式进行
  - 假设所有节点都共享了一份一致的**词项到其ID**的映射表

并行任务

- 两类**并行任务**分配给两类机器：
  - **分析器(Parser)**
  - **倒排器(Inverter)**
- 将输入的文档集**分片(split)** (对应于BSBI/SPIMI算法中的**块**)
- **每个数据片都是一个文档子集**

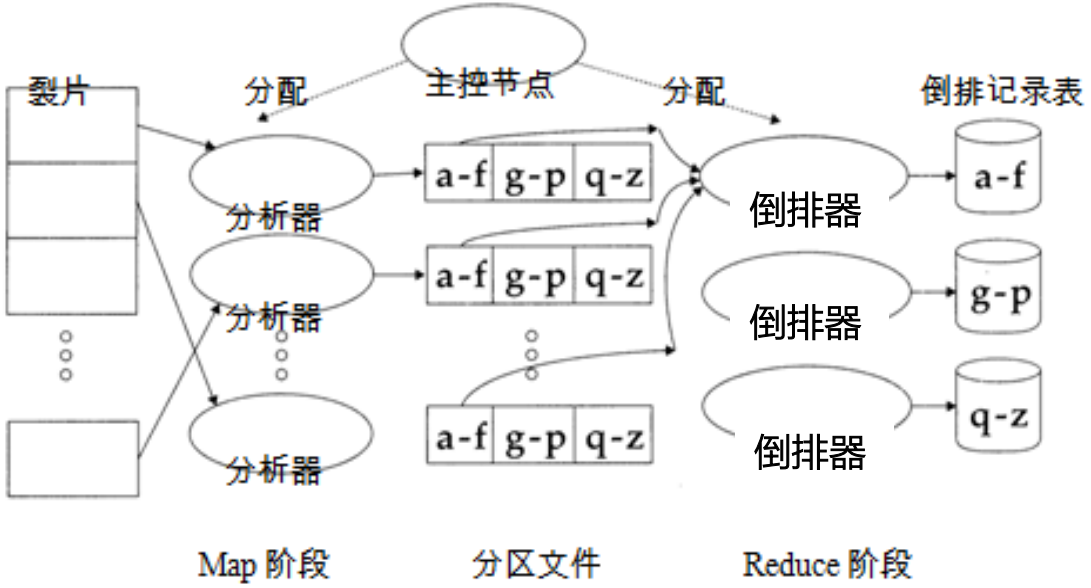
并行任务

- MapReduce的**Map过程将输入的数据片映射成键-值对**
- 映射过程对应于BSBI和SPIMI算法中的**分析任务**，因此也将执行**Map过程**的机器称为**分析器（parser）**
  - 每个分析器将输出结果存在本地的**中间文件**（也称为分区文件，segment file）中

分析器（Parser）

- 思路
  - 主节点将一个数据片分配给一台空闲的分析器
  - **分析器**一次读一篇文档然后输出(term,docID)-对
  - 分析器将这些(term,docID)-对又分成 j 个**词项分区**
  - **每个分区按照词项首字母进行划分**
    - E.g., a-f, g-p, q-z (这里 $j = 3$)

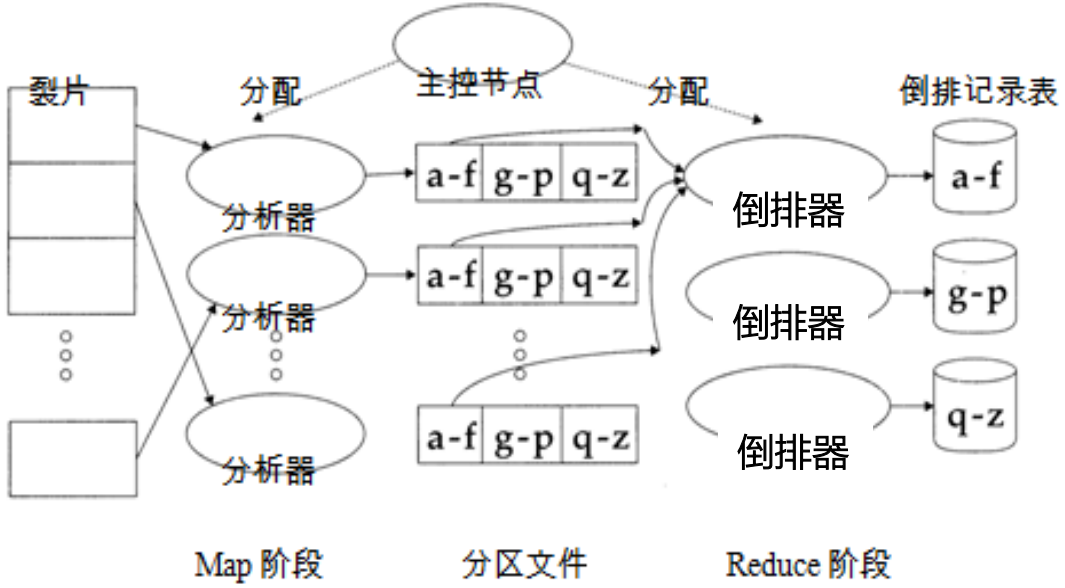
数据流



倒排器 (Inverter)

- 给定一个键（词项ID），将所有的值（文档ID）**汇总并组织成倒排表**的过程由**Reduce阶段**的**倒排器（inverter）**来完成

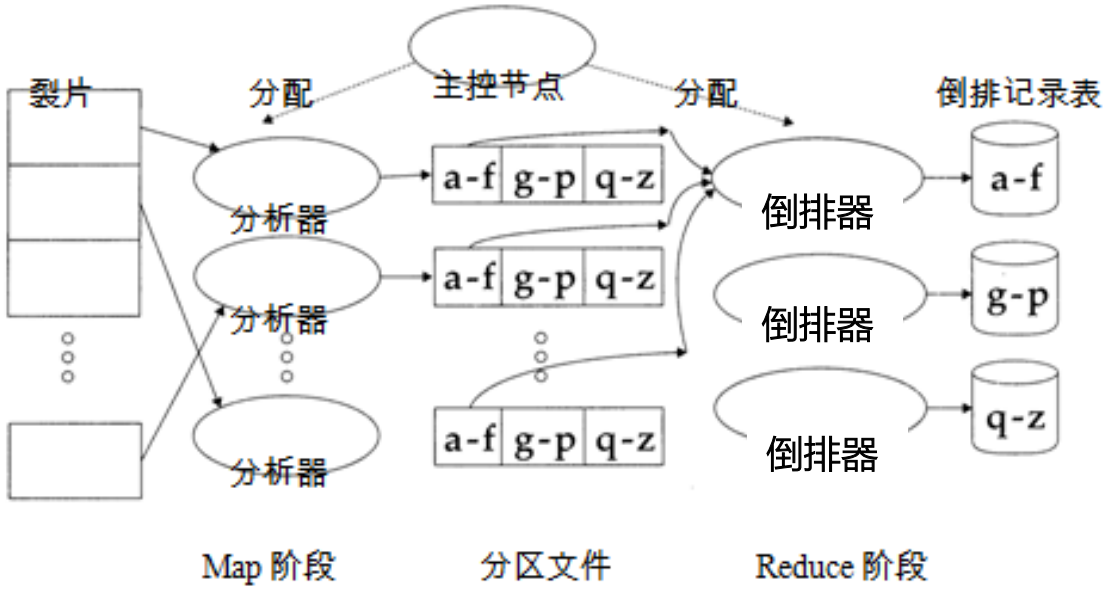
数据流



倒排器(Inverter)

- **总体思路**
  - **倒排器收集对应某一term分区(e.g., a-f分区)所有的 (term,docID)对(即倒排记录表)**
  - **排序并写进倒排记录表**

数据流

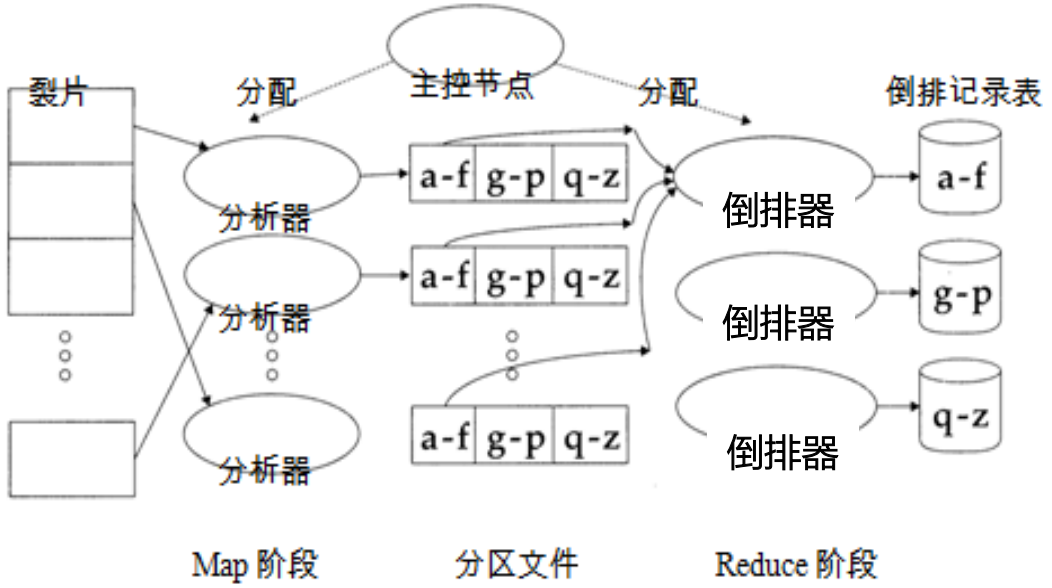


倒排器(Inverter)

- 具体步骤

1. 主控节点将每个词项分区分配给一个不同的倒排器，并在倒排器失效或者变慢的时候将在其上处理的词项分区进行重新分配
2. 每个词项分区（对应r个分区文件，其中每个文件存放在一个分析器上）由一个倒排器来完成。**假定多个分区文件的大小适合在单机上处理**

数据流



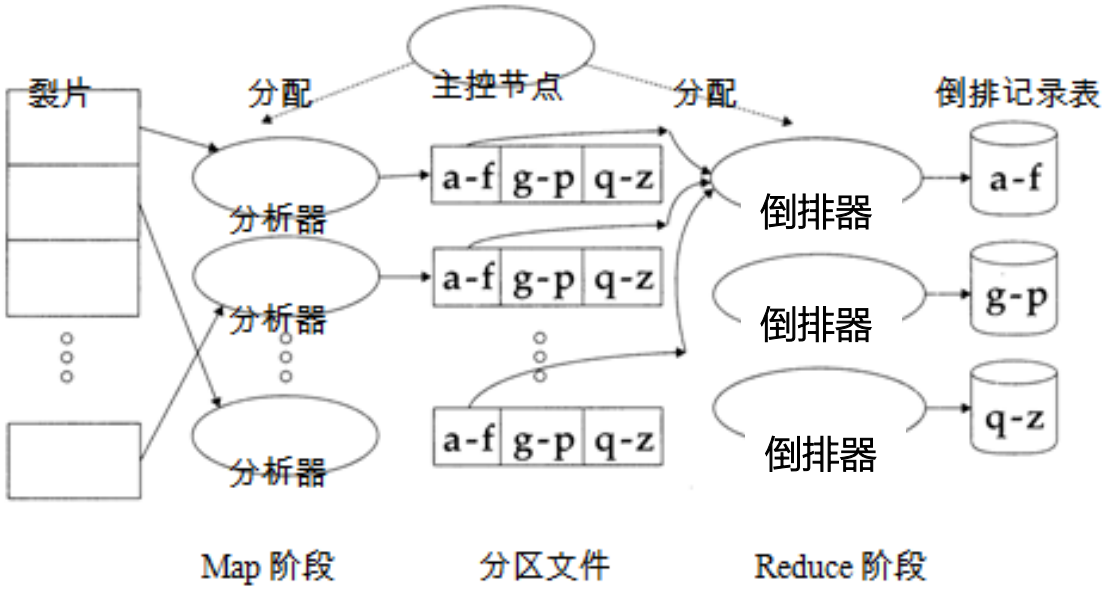
倒排器(Inverter)

- 具体步骤

3. 最后，每个键对应的所有值要进行**排序**并写到最终的排序倒排记录表（图中以“ 倒排记录表”来表示）中

到这里为止，整个倒排索引的构建才宣告完成

数据流



基于MapReduce的索引构建

Map和Reduce函数的构架

Map: 输入 → list(k, v)

Reduce: (k,list(v)) → 输出

索引构建中上述构架的实例化

Map: Web文档集 → list(词项ID, 文档ID)

Reduce: (〈文档ID$_1$, list(docID)〉, 〈文档ID$_2$, list(docID)〉, …) → (倒排记录表1, 倒排记录表2,…)

索引构建的一个例子

Map: d_2: C died. d_1: C came, C c'ed. → (〈C,d_2〉 〈died,d_2〉,〈C,d_1〉 〈came,d_1〉 〈C,d_1〉,〈c'ed,d_1〉)

Reduce: (〈C,(d_2,d_1,d_1)〉,〈died, (d_2)〉,〈came, (d_1)〉,〈c'ed,(d_1)〉) → (〈C,(d_1:2,d_2:1)〉,〈died, (d_2:1)〉,〈came,(d_1:1)〉, 〈c'ed,(d_1:1)〉)

MapReduce

- **分析器和倒排器并不一定是不同的机器**
  - 主控节点发现空闲的机器后会给它分配新的任务
  - **同一台机器在Map阶段中可以作为分析器，而在Reduce阶段也可以作为倒排器**
- 索引构建的同时，机器上往往也在**同时运行其他任务**，所以在做分析器和倒排器之外，一台机器也可能运行采集程序或者其他不相关的任务

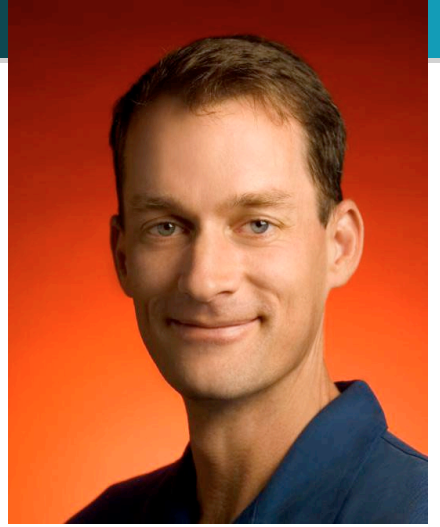
MapReduce

- 为了尽量减少在倒排器对数据进行Reduce之前的写时间，每个分析器都将其**分区文件写到本地磁盘**
- 在Reduce阶段，主控节点会通知倒排器与之相关的分区文件的位置（例如词项a～f分区对应的r个分区文件）
- 在每个分析器上，由于与某个特定倒排器相关的数据已经被分析器写入一个单独的分区文件中，所以每个分区文件仅需要**一次顺序读取过程**
  - 这种设置方法可以使索引时所需的网络通信开销最小

MapReduce总结

- 该**索引构建过程**实际上是**MapReduce的一个实例**
- **MapReduce是一个鲁棒的简单分布式计算框架**
  - 不一定需要在分布式处理的部分编写代码
- Google索引构建系统 (ca. 2002)
  - 多个步骤组成，每个步骤都采用**MapReduce**实现
  - 索引构建只是一个步骤
  - 另一个步骤：将**按词项分割**索引转换成**按文档分割**的索引

Jeff Dean（也称Jeffrey Adgate Dean）



是一名美国计算机科学家和软件工程师，出生于1968年，毕业于华盛顿大学

主要作品有《MapReduce》等。

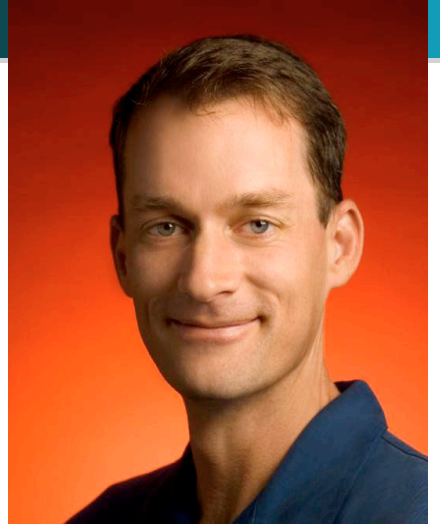
1990年，从明尼苏达大学计算机与经济学系取得学士学位，并为最优等（summa cum laude）。

1990-1991年，为世界卫生组织的艾滋病全球方案（World Health Organization's Global Programme on AIDS）开发软件，用于HIV传播的统计建模、预测和分析。

1996年，从华盛顿大学的计算机科学方向获得博士学位（PhD），研究方向为面向对象语言的程序优化。

1996-1999年，在DEC公司的Western Research Lab工作。

1999至今，在Google公司工作。



主要成就

MapReduce [2] 一个大规模数据处理系统

BigTable [3] 一个结构化数据的分布式存储系统

Spanner [4] 一个全球性的数据库系统

参与谷歌的网络爬虫、索引、查询系统、AdSense、谷歌翻译的设计与实现。

2009年当选美国工程院院士

TensorFlow [5]人工智能学习系统 项目负责人

ACM 计算奖（ACM Prize in Computing）

提纲

1. 上一讲回顾
2. 简介
3. BSBI算法
4. SPIMI算法
5. 分布式索引构建
6. 动态索引构建

动态索引构建

- 到目前为止，我们都假定**文档集是静态的**
- 实际中该假设很少成立：**文档会增加、删除和修改**
- 这也意味着**词典和倒排记录表必须要动态更新**

动态索引构建: 最简单的方法

- 构建
  - 在磁盘上维护一个大的**主索引(Main index)**
  - 新文档放入内存中较小的**辅助索引（Auxiliary index）**中
  - 同时搜索两个索引，然后合并结果
  - **定期将辅助索引合并到主索引中**
- 删除的处理
  - 采用无效位向量(Invalidation bit-vector)来表示删除文档
  - 利用该维向量过滤返回的结果，以去掉已删除文档

主辅索引合并中的问题

- **合并可能过于频繁**
- 合并时如果正好在搜索，那么搜索的性能将很低
- 实际上
  - 如果每个倒排记录表都采用一个单独的文件来存储的话，那么将辅助索引合并到主索引的代价并没有那么高
  - **此时合并等同于一个简单的添加操作，**但是这样做将需要大量的文件，效率显然不高
- 如果没有特别说明，本讲后面都**假定索引是一个大文件**
- **现实当中常常介于上述两者之间(例如：将大的倒排记录表分割成多个独立的文件，将多个小倒排记录表存放在一个文件当中……)**

对数合并(Logarithmic merge)

- **对数合并算法能够缓解(随时间增长)索引合并的开销**
  - → 用户并不感觉到响应时间上有明显延迟
- **维护一系列索引，其中每个索引是前一个索引的两倍大小**
- **将最小的索引 (Z_0) 置于内存**
- **其他更大的索引 ($I_0, I_1, \ldots$) 置于磁盘**
- 如果 Z_0 变得太大 ($> n$), 则将它作为 I_0 写到磁盘中(如果 I_0 不存在)
- 或者和 I_0 合并(如果 I_0 已经存在)， 并将合并结果作为I_1写到磁盘中(如果 I_1 不存在)， 或者和I_1合并(如果 I_0 已经存在)，以此类推……

```
LMergeAddToken(indexes, Z0, token)
 1  Z0 ← Merge(Z0, {token})
 2  if |Z0| = n
 3    then for i ← 0 to ∞
 4         do if Ii ∈ indexes
 5               then Zi+1 ← Merge(Ii, Zi)
 6                    (Zi+1 is a temporary index on disk.)
 7                    indexes ← indexes − {Ii}
 8               else Ii ← Zi      (Zi becomes the permanent index Ii.)
 9                    indexes ← indexes ∪ {Ii}
10                    Break
11         Z0 ← ∅

LogarithmicMerge()
1  Z0 ← ∅      (Z0 is the in-memory index.)
2  indexes ← ∅
3  while true
4  do LMergeAddToken(indexes, Z0, getNextToken())
```

对数合并-复杂度

- **索引数目的上界为 $O(\log T)$ (T 是所有倒排记录的个数)**
  - 因此，查询处理时需要合并$O(\log T)$个索引
- **索引构建时间为 $O(T \log T)$**
  - 这是因为每个倒排记录需要合并$O(\log T)$次
- **辅助索引方式： 索引构建时间为$O(T^2)$**
  - 因为每次合并都需要处理每个倒排记录
  - 假定每个辅助索引的大小为 a

$$a + 2a + 3a + 4a + \ldots + na = a\frac{n(n+1)}{2} = O(n^2)$$

- 因此，**对数合并的复杂度比辅助索引方式要低一个数量级**

对数合并(Logarithmic merge)-小结

- 在获得索引构建效率提升的同时，以**降低查询处理的效率**为代价
  - 这是因为，现在要合并的不止是主索引和辅助索引这两个索引的搜索结果，而是log2（T/n）个索引的结果
- 同辅助索引机制一样，这种机制偶尔也需要进行**大规模索引合并操作**，这会**降低在合并时的搜索效率**
  - 这种情况发生频率较低
  - 平均而言，合并中的索引规模也较小
- **同时拥有多个索引会增加全局统计信息维护的复杂性**
  - 例如，它会影响拼写校正算法（3.3 节），该算法基于最高选中次数选择最有可能的正确结果。在有多个索引和无效位向量的情况下，得到正确的词项选中数目不再是一个简单的查找过程

对数合并(Logarithmic merge)-小结

- 实际上，采用对数合并方法，信息检索系统的各个方面，包括索引维护、查询处理、分布等等，都要**复杂得多**
- **由于动态索引构建的复杂性，一些大型搜索引擎采用从头开始重构索引而不是动态构建索引的方法**
  1. **周期性地构建一个全新的索引**
  2. **将查询处理转到新索引**
  3. **同时将旧索引删除**

其他索引

<be : 993427;
1 : 7, 18, 33, 72, 86, 231;
2 : 3, 149;
4 : 17, 191, 291, 430, 434;
5 : 363, 367, ...>

- 本章前面部分仅仅讨论了**不包含位置信息**的索引构建
- 而**包含位置信息的索引结构**与无位置信息的索引结构相比
  - 考虑加入位置信息带来的更大数据开销
  - （词项ID，文档ID）二元组 → **（词项ID，文档ID，（位置1，位置2，...））三元组**
  - 也就是**每个文档ID后面附有词项ID在文档中的位置信息**
  - 这样一来，这里所讨论的所有算法都能应用到带位置信息的索引结构上

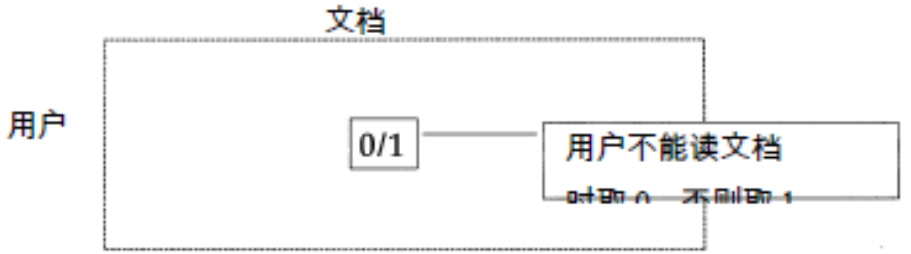
其他索引

- 在目前已讨论过的索引结构中，**倒排记录表中的所有记录会按照文档ID排序**
- 这样做会对**压缩**提供方便，即可以不直接保存文档ID而是**保存文档ID之间的间隔**来进行压缩，这样就能够减少索引的存储空间
- 然而，这种结构对于**排序式检索（ranked retrieval）系统（非布尔检索系统）**而言却不是最优的
  - 在排序式检索系统中，倒排记录往往基于**权重或者影响度**排序，其中**权重最大的倒排记录排在首位**
  - 在这种结构下进行查询时，并不一定需要对所有的倒排记录进行扫描，而是在遇到**权重足够小的文档后即可停止**，因为后面的文档和查询的相似度可能都较低

其他索引

- **以文档ID排序的索引（传统布尔检索）**
  - 新文档的加入往往只需在**倒排记录表最后增加信息**
- **以文档影响度排序**的索引(**排序式检索**)
  - 新文档的加入会导致在**任意可能的地方插入信息**
  - 会大大增加倒排表更新的复杂度

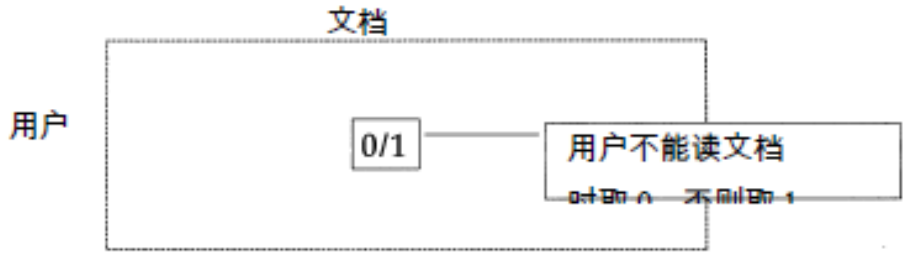
其他索引



- **安全性是很多企业检索系统所必须重点关注的问题**
  - 一个低级别的员工不能对公司所有人的工资进行搜索，而只有授权的员工才能这样做
  - 用户的搜索结果中不能出现禁止公开的文档，文档的存在本身也可能就是敏感信息
- 用户授权往往通过**ACL（access control list，访问控制表）**来实现
- 对于信息检索系统来说，ACL 可以通过将**每篇文档表示成所有能访问该文档的用户集合**来实现，然后再对结果用户—文档矩阵进行转置

- **用户—文档的访问控制矩阵**，如果用户*i*具有访问文档*j*的权限，那么矩阵元素（*i*, *j*）取1，否则取0
- 对查询进行处理时，**用户的访问控制倒排记录表将和采用文本倒排索引得到的结果求交集**

其他索引



- 转置后的ACL 索引中有一个每个用户都可以访问的所有文档组成的“ 倒排记录表”，这个表就是**用户的访问表**，搜索结果会和这个表进行**求交集运算**
- 然而，**维护这样的索引是比较困难的**，特别是在**访问权限发生变化**的时候
- 在4.5 节介绍常规倒排索引的增量式构建问题时，就提到了上述困难
  - 有些用户的可访问文档比较多，此时就要对长倒排记录表进行额外处理
  - 在查询时，用户的资格往往通过直接从文件系统返回的访问信息来验证，即使这样会降低检索的速度

本讲内容

- 计算机硬件基础知识
- 两种索引构建算法
  - BSBI (简单)
  - SPIMI (更符合实际情况)
- 分布式索引构建: MapReduce
- 动态索引构建: 如何随着文档集变化更新索引
- 其他索引的讨论

参考资料

- 《信息检索导论》第4章
- http://ifnlp.org/ir
  - Dean and Ghemawat (2004) 有关MapReduce的原作
  - Heinz and Zobel (2003) 有关SPIMI的原作
  - YouTube视频: Google数据中心